

A PROMPT LIBRARY FOR CODING AGENTS

The Agentic Coding Prompt Library

135 ready-to-use prompts for Claude Code, Codex, Cursor and other coding agents

135 prompts • 15 categories

Planning • Building • Debugging • Refactoring • Testing • Review • Docs • Performance • Security • Migrations • DevOps • Automation

How to use this library

Every prompt below is written to be pasted straight into a coding agent that can read your repository, run commands and edit files. Anything in [SQUARE BRACKETS] is a placeholder: replace it with your file path, framework, ticket number or error message before you send it. Prompts are grouped by the job you are actually doing, so you can jump to the section you need and ignore the rest.

- 01** Replace every [PLACEHOLDER] before sending. An unfilled placeholder is the single most common reason an agent guesses wrong.
 - 02** Give the agent a way to check its own work: a test command, a build, a script it can run. Without that it stops as soon as the code looks finished.
 - 03** Scope the prompt to specific files or directories when you know them. A narrow scope produces far better results than a whole-repo instruction.
 - 04** Ask for a plan before edits on anything non-trivial. Review the plan, correct it, then let the agent execute.
 - 05** Keep persistent project context in the agent's rules file (CLAUDE.md, AGENTS.md, .cursor/rules) so you do not repeat it in every prompt.
-

These prompts are written to be tool-agnostic. They work with Claude Code, Codex, Cursor, and any other agent that can read a repository and run commands. Where a prompt refers to a rules file, use whichever your tool expects.

Contents

- 01 Codebase Onboarding and Understanding** Prompts 1 to 10
- 02 Planning and Architecture** Prompts 11 to 20
- 03 Building New Features** Prompts 21 to 30
- 04 Debugging and Root Cause Analysis** Prompts 31 to 40
- 05 Refactoring and Code Quality** Prompts 41 to 50
- 06 Testing** Prompts 51 to 60
- 07 Code Review and Pull Requests** Prompts 61 to 69
- 08 Git and Version Control** Prompts 70 to 78
- 09 Documentation** Prompts 79 to 87
- 10 Performance Optimization** Prompts 88 to 95
- 11 Security and Dependencies** Prompts 96 to 103
- 12 Migrations and Upgrades** Prompts 104 to 111
- 13 DevOps, CI and Infrastructure** Prompts 112 to 119
- 14 Data, APIs and Integrations** Prompts 120 to 127
- 15 Agent Setup, Rules and Automation** Prompts 128 to 135

SECTION 01

Codebase Onboarding and Understanding

Use these on day one in a new repository, or any time you inherit code nobody can explain.

1. Repository orientation tour

Use when: *First hour in an unfamiliar codebase.*

Give me an orientation to this repository. Cover: what the project does, the main entry points, how the code is organised at the top level, which directories hold the business logic versus configuration versus tests, and the three files a new contributor should read first. Do not list every file. Keep it under 400 words.

2. Trace a feature end to end

Use when: *You need to know where a behaviour actually lives.*

Trace how [FEATURE OR USER ACTION] flows through this codebase, from the entry point to the data layer and back. List each file and function it passes through in order, with a one-line description of what happens at each hop. Point out anything that surprised you.

3. Explain this file

Use when: *A single file is dense or undocumented.*

Read [FILE PATH] and explain what it does, who calls it, what it calls, and what would break if it were deleted. Flag any logic that looks like it was written for a reason that is no longer obvious.

4. Map the data model

Use when: *Before touching anything database related.*

Map the data model in this project. List every table or collection, its key fields, and the relationships between them. Then tell me which models are central to the domain and which look like leftovers.

5. Find the architectural boundaries

Use when: *Planning a refactor or a split.*

Identify the architectural boundaries in this codebase: which modules are meant to be independent, and where those boundaries are currently being violated by direct imports or shared state. Give me the specific import lines that cross a boundary they should not.

6. Locate the code for a ticket

Use when: *You have a ticket but not a file path.*

I need to work on this ticket: [TICKET DESCRIPTION]. Search the codebase and tell me exactly which files I will need to touch and why. Do not make any changes yet.

7. Explain an unfamiliar pattern

Use when: *The codebase uses a convention you do not recognise.*

This codebase uses [PATTERN OR LIBRARY] in [FILE PATH]. Explain the pattern, why a team might choose it here, and what I need to know to work with it without breaking the convention.

8. Summarise recent history

Use when: *Returning to a project after time away.*

Look at the git history for the last [NUMBER] weeks and summarise what changed: the main areas of work, any refactors in progress, and anything that looks half finished. Group by theme, not by commit.

9. Find the dead code

Use when: *Cleanup sprint.*

Find code in this repository that appears to be unused: exported functions with no callers, files nothing imports, feature flags that are permanently off, and commented-out blocks. For each, state your confidence that it is truly dead and what you checked.

10. Reconstruct undocumented behaviour

Use when: *The docs are missing or wrong.*

There is no documentation for [MODULE OR ENDPOINT]. Read the implementation and the tests, then write an accurate description of its actual behaviour, including edge cases and error responses. Mark anything where the tests and the implementation disagree.

SECTION 02

Planning and Architecture

Get the thinking right before a single line is written. These prompts should produce a plan you review, not code.

11. Plan before you code

Use when: Any task larger than a one-file change.

I want to implement [FEATURE]. Before writing any code, produce an implementation plan: the files you will create or modify, the order you will do it in, the interfaces between the pieces, and the tests you will add. Stop after the plan so I can review it.

12. Compare three approaches

Use when: The right design is not obvious.

I need to solve [PROBLEM] in this codebase. Propose three genuinely different approaches. For each, give the implementation sketch, what it costs in complexity, how it behaves under [CONSTRAINT], and what it makes harder later. Then recommend one and say why.

13. Design a new module

Use when: Adding a substantial new area of the system.

Design a new [MODULE NAME] module for this project that handles [RESPONSIBILITY]. Match the existing architectural conventions in this repo. Give me the file structure, the public interface, and how it will integrate with [EXISTING MODULE]. No implementation yet.

14. Write a technical design doc

Use when: Work that needs sign-off from others.

Write a technical design document for [FEATURE]. Include: problem statement, goals and explicit non-goals, proposed design, data model changes, API changes, migration and rollout plan, failure modes, and open questions. Write it for an engineer who has never seen this codebase.

15. Break work into shippable slices

Use when: A large project needs to be delivered incrementally.

Break [PROJECT] into a sequence of pull requests that can each be merged and deployed independently without breaking production. For each slice give the scope, the rough size, what it unblocks, and whether it needs a feature flag.

16. Interrogate the requirements

Use when: *The spec feels underspecified.*

Here is what I have been asked to build: [REQUIREMENT]. Before you propose anything, list the questions you would need answered to build this correctly, ordered by how much the answer would change the design. Then state the assumptions you would make if nobody answers.

17. Design the API contract first

Use when: *Frontend and backend are being built in parallel.*

Design the API contract for [FEATURE]. Give me the endpoints or operations, request and response shapes with types, status codes, error payloads, pagination behaviour, and versioning approach. Follow the conventions already used in [EXISTING API FILE].

18. Pressure-test a design

Use when: *You have a plan and want it attacked.*

Here is my plan for [FEATURE]: [PLAN]. Argue against it. What breaks at 100x the current load, what happens when [DEPENDENCY] is unavailable, what is hard to test, and what will the next engineer curse me for. Be specific, not generic.

19. Estimate the blast radius

Use when: *Before committing to a change.*

If I change [COMPONENT] to [NEW BEHAVIOUR], what else in this codebase is affected? List every caller, every test that will need updating, every config or migration involved, and anything outside the repo that depends on the current behaviour.

20. Choose a library

Use when: *Adding a dependency.*

I need [CAPABILITY] in this project, which uses [STACK]. Compare the realistic library options on: maintenance activity, bundle or install size, API ergonomics for our use case, and how hard it would be to replace later. Also tell me whether we could reasonably do it without a dependency.

SECTION 03

Building New Features

Implementation prompts. All of them assume the agent can read the surrounding code and match its conventions.

21. Implement from an approved plan

Use when: *The plan is agreed and you want execution.*

Implement the plan we agreed. Work through it in order, run [TEST COMMAND] after each meaningful step, and stop and tell me if reality differs from the plan rather than improvising around it.

22. Build a feature matching existing conventions

Use when: *Consistency matters more than novelty.*

Add [FEATURE] to this project. Before writing anything, read [SIMILAR EXISTING FEATURE PATH] and follow the same structure, naming, error handling and test style. Point out any place where following the existing convention would be a mistake.

23. Implement to a spec

Use when: *You have a written specification.*

Implement this specification exactly: [SPEC]. Do not add features that are not specified. Where the spec is ambiguous, list the ambiguity and pick the more conservative reading. Add tests that assert the specified behaviour.

24. Add an endpoint

Use when: *Extending an API.*

Add a [METHOD] [ROUTE] endpoint that [BEHAVIOUR]. Include input validation, authorisation matching the existing pattern, error responses, and tests for the success path plus at least three failure paths.

25. Build a UI component

Use when: *Frontend work.*

Build a [COMPONENT NAME] component in [FRAMEWORK] that [BEHAVIOUR]. Match the styling approach already used in [EXISTING COMPONENT PATH]. Handle loading, empty and error states. Make it keyboard accessible and give interactive elements accessible names.

26. Wire a form end to end

Use when: *Forms touch validation, state and the API at once.*

Wire up a form for [PURPOSE]: client-side validation, submission to [ENDPOINT], optimistic or pending state, server error display, and success handling. Reuse the validation schema on both sides if the stack allows it.

27. Add a background job

Use when: Work that must happen out of the request cycle.

Add a background job that [BEHAVIOUR], using the job system already in this project. Make it idempotent, give it sensible retry and backoff behaviour, log enough to debug a failure, and tell me how it should be scheduled.

28. Feature flag a change

Use when: Risky change that needs a kill switch.

Implement [FEATURE] behind a feature flag named [FLAG NAME], following the flag pattern already used in this codebase. Both code paths must work. Tell me exactly what I need to delete once the flag is fully rolled out.

29. Add an integration with a third-party service

Use when: Talking to an external API.

Integrate [SERVICE] into this project to [PURPOSE]. Put the client behind an interface so it can be faked in tests, handle rate limits and timeouts explicitly, never log credentials, and read configuration the same way the rest of the project does.

30. Finish someone else's half-built feature

Use when: Picking up abandoned work.

There is a partially implemented [FEATURE] in [PATH]. Work out what was intended, what is done, and what is missing. Give me that assessment first. Then finish it in the style it was started, unless the existing approach is broken, in which case say so.

SECTION 04

Debugging and Root Cause Analysis

The rule for all of these: make the agent find the cause and prove it before it proposes a fix.

31. Root cause, not symptom

Use when: Default debugging prompt.

This is failing: [ERROR OR BEHAVIOUR]. Find the root cause before proposing any fix. Show me the specific line where it goes wrong and explain the chain that leads there. Then propose a fix that addresses the cause rather than the symptom.

32. Reproduce it first

Use when: The bug is intermittent or unclear.

Write the smallest possible failing test that reproduces [BUG]. Run it and confirm it fails for the right reason. Only then fix the code and show me the test passing.

33. Decode a stack trace

Use when: You have an error and nothing else.

Here is a stack trace from production: [PASTE TRACE]. Work out what happened, which of our code is implicated versus library code, what input would produce it, and what the fix is. Say clearly if the trace is not enough to be sure.

34. Bisect the regression

Use when: It used to work.

[BEHAVIOUR] used to work and now does not. Use git history to find the commit that most likely broke it. Explain the mechanism by which that commit caused this, and confirm by reading the diff rather than guessing from the message.

35. Debug by hypothesis

Use when: Nothing obvious explains it.

I am seeing [SYMPTOM]. List the possible causes ranked by likelihood. For each, tell me the cheapest check that would confirm or eliminate it. Then run the checks you can run yourself and report what you found.

36. Fix a flaky test

Use when: CI fails at random.

The test [TEST NAME] fails intermittently. Find the source of non-determinism: shared state, timing, ordering, real clocks, real network, random data. Fix the cause. Do not fix it by adding a sleep or a retry.

37. Chase a memory or resource leak

Use when: *Usage grows over time.*

This process leaks [MEMORY / CONNECTIONS / FILE HANDLES] over time. Look for unbounded caches, listeners that are never removed, resources opened without cleanup, and closures holding references longer than intended. Show me the evidence for each candidate.

38. Explain the concurrency bug

Use when: *Rare, unreproducible failures.*

We occasionally see [SYMPTOM] under load. Analyse [PATH] for race conditions, unsafe shared state, missing locks, and non-atomic read-modify-write sequences. Describe the exact interleaving that produces the failure.

39. Compare working and broken environments

Use when: *Works locally, fails in CI or production.*

[BEHAVIOUR] works in [ENVIRONMENT A] and fails in [ENVIRONMENT B]. Compare the configuration, dependency versions, environment variables and runtime assumptions between the two. Give me the differences that could plausibly explain it, ranked.

40. Add debugging instrumentation

Use when: *You cannot reproduce it and need data.*

I cannot reproduce [BUG] locally. Add temporary logging or instrumentation that would let me diagnose it from production output. Log the values that would actually discriminate between the possible causes, not everything. Mark clearly what to remove afterwards.

SECTION 05

Refactoring and Code Quality

Refactors should not change behaviour. Every prompt here asks the agent to prove that.

41. Refactor without changing behaviour

Use when: Standard refactor prompt.

Refactor [FILE OR FUNCTION] to be clearer, without changing its observable behaviour. Run [TEST COMMAND] before and after and confirm the results are identical. If the tests are too thin to prove that, tell me before you start.

42. Break up a large file

Use when: A file has grown past comprehension.

[FILE PATH] is too large. Propose a split into coherent modules, with each module's responsibility in one sentence. Show me the proposed structure before moving anything. Preserve the public interface exactly.

43. Extract the duplicated logic

Use when: The same code exists in several places.

Find the duplicated logic across [PATHS] and extract it into a shared implementation. Be careful: only unify code that is duplicated for the same reason. Where two copies look similar but serve different purposes, leave them alone and tell me why.

44. Simplify the conditionals

Use when: Deeply nested branching.

The control flow in [FUNCTION] is hard to follow. Simplify it using early returns, guard clauses and extracted predicates with meaningful names. Do not change what it does. Show me the before and after side by side.

45. Improve the naming

Use when: The code is correct but unreadable.

Review the naming in [FILE PATH]. Point out names that are vague, misleading, inconsistent with the rest of the codebase, or that describe implementation rather than intent. Propose better names and apply them consistently including callers.

46. Make error handling consistent

Use when: Every function fails differently.

Error handling in [PATH] is inconsistent. Map how errors are currently produced and handled, propose a single consistent approach that fits this codebase, then apply it. Errors must carry enough context to be debugged from a log line alone.

47. Tighten the types

Use when: *Typed language, loose types.*

Strengthen the types in [FILE PATH]. Replace any and unknown escape hatches with real types, make illegal states unrepresentable where that is reasonable, and narrow overly wide parameter types. Do not change runtime behaviour.

48. Remove a deprecated pattern

Use when: *Migrating away from an old approach.*

This codebase still uses [DEPRECATED PATTERN] in places. Find every occurrence and convert it to [NEW PATTERN]. Work file by file, run the tests after each, and list any occurrence that cannot be converted mechanically.

49. Reduce the coupling

Use when: *Everything depends on everything.*

[MODULE] is too tightly coupled to [OTHER MODULE]. Show me every point of coupling, then propose the smallest change that would let them evolve independently. Prefer an interface over an event bus unless there is a reason.

50. Clean up before handover

Use when: *Prepping code for someone else.*

Prepare [PATH] for handover to another engineer. Remove dead code and stale comments, fix misleading names, add comments only where the code cannot explain itself, and write a short note at the top of the module explaining its role. Do not change behaviour.

SECTION 06

Testing

Ask for tests that would actually fail if the code were wrong.

51. Write meaningful tests

Use when: New or untested code.

Write tests for [FILE OR FUNCTION]. Cover the happy path, the boundary conditions, and the failure modes. Each test should fail if the behaviour it describes is broken. Do not write tests that only assert the code runs.

52. Find the gaps in coverage

Use when: Coverage numbers look fine but bugs get through.

Look at [MODULE] and its existing tests. Tell me which real behaviours are untested, not which lines are unexecuted. Rank the gaps by how much damage a bug there would do.

53. Test the edge cases

Use when: Hardening existing code.

List the edge cases for [FUNCTION]: empty input, maximum size, nulls, unicode, concurrent calls, clock changes, and anything else specific to this domain. Write a test for each one that is currently unhandled, and tell me which ones the code fails.

54. Add a regression test

Use when: Immediately after fixing a bug.

We just fixed [BUG]. Add a regression test that would have caught it. Confirm the test fails against the pre-fix behaviour and passes now. Name it so the next engineer understands what it protects.

55. Write an integration test

Use when: Unit tests pass but the system does not work.

Write an integration test that exercises [WORKFLOW] across real component boundaries. Use the existing test setup in [TEST PATH]. Fake only what is genuinely external. Make sure it fails for real reasons, not setup problems.

56. Property-based tests

Use when: Logic with invariants.

Write property-based tests for [FUNCTION] using [LIBRARY]. Identify the properties that must always hold regardless of input, generate inputs across the full domain, and show me any counterexamples the generator finds.

57. Refactor the test suite

Use when: *Tests are slow, brittle or unreadable.*

Review the tests in [TEST PATH]. Identify tests that are brittle, that duplicate each other, that test implementation instead of behaviour, or that would pass even if the code were broken. Propose a cleanup plan before changing anything.

58. Speed up the test suite

Use when: *CI takes too long.*

The test suite takes [DURATION]. Find the slowest tests and the reasons they are slow: real network calls, real sleeps, oversized fixtures, unnecessary setup, missing parallelism. Fix the ones you can without weakening coverage.

59. Test the error paths

Use when: *Only the happy path is covered.*

The tests for [MODULE] cover the success cases only. Add tests for every failure mode: invalid input, dependency unavailable, timeout, partial failure, permission denied. Assert the specific error behaviour, not just that something threw.

60. Build realistic fixtures

Use when: *Test data does not resemble production data.*

The test fixtures in [PATH] are unrealistically clean. Build fixtures that resemble real production data, including the messy cases: missing optional fields, legacy formats, long strings, non-Latin characters. Then tell me which tests break.

SECTION 07

Code Review and Pull Requests

Use an agent as a first reviewer so humans spend their attention on the things that need judgment.

61. Review my changes before I push

Use when: Every time, ideally.

Review my uncommitted changes. Look for bugs, missed edge cases, security issues, inconsistency with the existing codebase, and anything I have left in by accident such as debug output or commented code. Be direct. Skip style points the linter already catches.

62. Review a pull request

Use when: Reviewing someone else's work.

Review the diff for [BRANCH OR PR]. For each issue tell me the file and line, the severity, why it matters, and the suggested fix. Separate blocking issues from suggestions. Also say what the change does well.

63. Ask what the author forgot

Use when: The code looks fine.

This change implements [FEATURE]. Ignore whether the code is correct and tell me what is missing around it: tests, migrations, documentation, monitoring, error handling, feature flags, rollback path, and anything else this codebase normally expects with a change like this.

64. Review for a specific concern

Use when: Targeted review.

Review [FILES] specifically for [CONCERN: security / performance / accessibility / concurrency / backwards compatibility]. Ignore everything else. Give me findings ranked by severity with concrete fixes.

65. Write the pull request description

Use when: The work is done.

Write a pull request description for the changes on this branch. Cover what changed and why, the approach and any alternatives rejected, how to test it, risks, and anything a reviewer should look at closely. Read the actual diff rather than relying on commit messages.

66. Explain a diff you did not write

Use when: Reviewing unfamiliar code.

Explain what this diff does and why the author probably did it this way: [PASTE DIFF OR BRANCH NAME]. Highlight anything that changes behaviour in a way the description does not mention.

67. Check consistency with the codebase

Use when: *New contributor or new pattern.*

Does this change follow the conventions of this codebase? Compare it against similar existing code for structure, naming, error handling, testing approach and file placement. List every deviation and say whether it is an improvement or an inconsistency.

68. Second-guess the review

Use when: *You disagree with feedback.*

I received this review comment: [COMMENT]. Here is the code: [CODE OR PATH]. Assess whether the comment is right, partly right, or based on a misunderstanding. Give me the strongest version of the reviewer's argument, then your assessment.

69. Pre-merge safety check

Use when: *Just before merging something risky.*

This branch is about to merge. Check for: breaking changes to public interfaces, database migrations that are not backwards compatible, changed environment variables or config, removed endpoints, and anything that requires a deploy step. List what needs to happen at deploy time.

SECTION 08

Git and Version Control

The agent can read history, which makes it good at everything from commit messages to untangling a bad merge.

70. Commit my work sensibly

Use when: You have a pile of changes.

Look at my uncommitted changes and group them into logical commits. Propose the grouping and the commit messages first. Follow the commit message style already used in this repository.

71. Write a commit message

Use when: Staged and ready.

Write a commit message for the staged changes. Explain what changed and why, not how. Match the existing convention in this repo for format and prefix.

72. Split a commit

Use when: One commit does too many things.

Commit [HASH] mixes [THING A] and [THING B]. Split it into separate commits that each build and pass tests independently. Walk me through the steps rather than doing anything destructive without confirmation.

73. Resolve a merge conflict

Use when: Conflict markers everywhere.

Resolve the merge conflicts in [FILE]. Read both sides and understand the intent of each before choosing. Where both changes are needed, combine them correctly rather than picking one. Explain each resolution and flag any you are unsure about.

74. Write release notes from history

Use when: Shipping a version.

Generate release notes for the changes between [TAG A] and [TAG B]. Group by user-visible features, fixes, and internal changes. Write for users, not for developers. Call out anything that requires action on upgrade.

75. Investigate why a line exists

Use when: Tempted to delete something odd.

Why does [FILE PATH] line [NUMBER] do what it does? Use git blame and the surrounding commit history to find the change that introduced it and the context around it. Tell me whether it is still needed.

76. Clean up a branch before review

Use when: *Messy work-in-progress history.*

My branch has [NUMBER] messy commits. Propose a cleaned-up history: squash the fixup commits, keep the logically distinct steps separate, and write proper messages. Show me the plan before rewriting anything.

77. Recover from a bad git state

Use when: *Something went wrong.*

I ran [COMMAND] and now [PROBLEM]. Tell me the current state of the repository, what I can recover and how, and the exact commands to run. Warn me clearly before anything destructive and tell me what to back up first.

78. Compare two branches

Use when: *Before merging long-lived branches.*

Compare [BRANCH A] and [BRANCH B]. Summarise the functional differences rather than listing the diff, tell me where they conflict, and identify anything on one branch that would be silently lost by merging the other.

SECTION 09

Documentation

Documentation written from the code is more accurate than documentation written from memory.

79. Write the README

Use when: There is no README, or it is stale.

Write a README for this project based on what the code actually does. Cover: what it is, prerequisites, install and run steps, configuration, the common development commands, and how to run the tests. Verify every command you list actually exists in this repo.

80. Document a module

Use when: Onboarding cost is high.

Write documentation for [MODULE]. Cover its purpose, the public interface with parameters and return values, usage examples taken from real calls in this codebase, error behaviour, and the gotchas someone would otherwise discover the hard way.

81. Generate API documentation

Use when: An API exists but is not documented.

Generate API documentation for the endpoints in [PATH]. For each: method, route, auth requirements, parameters with types and whether they are required, example request, example success response, and every error response with its status code. Read the implementation, do not infer from names.

82. Write a runbook

Use when: On-call handover.

Write an operational runbook for [SERVICE]. Cover: what it does, its dependencies, how to tell if it is healthy, the common failure modes with symptoms and fixes, how to restart it safely, and who or what to escalate to.

83. Add comments where they help

Use when: Code with non-obvious reasoning.

Add comments to [FILE PATH] only where the code cannot explain itself: non-obvious reasoning, deliberate deviations from the obvious approach, workarounds for external bugs, and performance-motivated ugliness. Do not narrate what the code plainly says.

84. Write an architecture overview

Use when: *New engineers keep asking the same questions.*

Write an architecture overview of this system: the components, what each is responsible for, how they communicate, where data lives, and what the main request or data flows look like. Include a text diagram. Aim for something a new engineer could read in ten minutes.

85. Document a decision

Use when: *You made a choice worth recording.*

Write an architecture decision record for the choice to [DECISION]. Cover the context, the options considered, the decision, the reasoning, and the consequences including the ones we will not like. Keep it to one page.

86. Write a migration guide

Use when: *You changed something consumers depend on.*

Write an upgrade guide for consumers moving from [OLD VERSION] to [NEW VERSION]. List every breaking change, the before and after code for each, and a suggested order of operations. Be explicit about anything that cannot be automated.

87. Update stale documentation

Use when: *The docs and the code have drifted.*

Compare the documentation in [DOC PATH] against the current implementation. List every statement that is now wrong, outdated or missing. Then correct the document. Flag anything where you cannot tell which one is right.

SECTION 10

Performance Optimization

Measure first. Every one of these prompts asks for evidence before changes.

88. Profile before optimising

Use when: Something is slow and you do not know why.

[OPERATION] is slow. Before changing anything, work out where the time actually goes. Profile it or add timing instrumentation, and show me the breakdown. Then tell me which single change would have the largest effect.

89. Find the database bottleneck

Use when: Slow endpoints, healthy CPU.

Analyse the database access in [PATH]. Look for N+1 queries, missing indexes, queries selecting more than they need, unnecessary round trips, and transactions held open too long. For each finding, show me the query and the fix.

90. Optimise a hot path

Use when: You know which code is hot.

[FUNCTION] is on a hot path called [FREQUENCY]. Optimise it. Preserve exact behaviour, keep it readable unless the gain is large enough to justify otherwise, and tell me the expected improvement and how to verify it.

91. Reduce bundle size

Use when: Frontend load times.

Analyse the bundle for this project. Show me the largest contributors, which dependencies are pulled in for small amounts of functionality, what is not being tree-shaken, and what could be lazily loaded. Rank fixes by bytes saved versus effort.

92. Add caching correctly

Use when: Repeated expensive work.

Add caching for [OPERATION]. Tell me first what the cache key should be, what the invalidation rule is, and what goes wrong if the cache is stale. Only implement once we agree the invalidation is correct.

93. Improve perceived performance

Use when: You cannot make it faster.

[OPERATION] cannot easily be made faster. Suggest ways to improve how fast it feels: optimistic updates, streaming or progressive rendering, skeleton states, prefetching, background processing with notification. Assess each for this specific case.

94. Find the memory hogs

Use when: *Memory pressure.*

Analyse memory usage in [PATH]. Find the largest allocations, data structures held longer than needed, duplicated data in memory, and anything loading a full dataset when it could stream. Show the evidence for each.

95. Set a performance budget

Use when: *Preventing future regressions.*

Propose performance budgets for this project: response time targets for the key operations, bundle size limits, and query count limits. Base them on current measured performance, then tell me how to enforce them in CI.

SECTION 11

Security and Dependencies

Defensive work only. These prompts look for weaknesses in your own code so you can close them.

96. Security review of a change

Use when: Anything touching auth, input or data.

Review [FILES] for security issues: unvalidated input, injection risks, authorisation checks that are missing or in the wrong place, secrets in code or logs, unsafe deserialisation, and information leaked through error messages. Rank by severity with concrete fixes.

97. Audit the authorisation logic

Use when: Multi-user or multi-tenant systems.

Audit the authorisation logic in this codebase. For each protected resource, show me where the check happens and whether it can be bypassed by a different code path. Pay particular attention to endpoints that take an ID from the user.

98. Hunt for exposed secrets

Use when: Before open-sourcing or after an incident.

Search this repository for credentials, API keys, tokens, private keys, connection strings and other secrets, including in config files, tests, fixtures and git history. For each finding, state whether it looks real and what should be done about it.

99. Validate all the inputs

Use when: Public-facing surfaces.

Review every input to [MODULE] from an untrusted source. For each, tell me whether it is validated, where, and whether the validation is sufficient. Add validation where it is missing, failing closed rather than open.

100. Review dependencies

Use when: Periodic hygiene.

Review this project's dependencies. Identify: known-vulnerable versions, packages that are unmaintained, packages pulling in far more than we use, and anything with a suspiciously broad set of permissions or install scripts. Recommend an action for each.

101. Plan a safe dependency upgrade

Use when: Updates are overdue.

Our dependencies are [NUMBER] versions behind. Group them into: safe patch updates, minor updates needing a test run, and major updates needing real work. Give me an order to do them in that minimises the chance of a confusing failure.

102. Check the logging for leaks

Use when: Handling personal or sensitive data.

Review everything this application logs. Flag anywhere personal data, credentials, tokens, full request bodies or other sensitive values could reach the logs, including inside error objects and stack traces. Show me the fix for each.

103. Harden the configuration

Use when: Before a production launch.

Review the configuration and deployment setup for insecure defaults: debug mode, permissive CORS, missing security headers, overly broad permissions, unencrypted connections, default credentials. List what to change and why each matters.

SECTION 12

Migrations and Upgrades

Large mechanical changes are exactly what agents are good at, provided the plan is right.

104. Plan a framework upgrade

Use when: Major version bump ahead.

Plan an upgrade from [CURRENT VERSION] to [TARGET VERSION] of [FRAMEWORK] for this project. Read the release notes, then list every breaking change that affects our code with the specific files involved. Order the work so the app stays runnable throughout.

105. Execute a mechanical migration

Use when: Same change, many files.

Convert every occurrence of [OLD PATTERN] to [NEW PATTERN] across this codebase. Do it in batches, run [TEST COMMAND] after each batch, and stop to ask when a case does not fit the mechanical rule.

106. Write a database migration

Use when: Schema change.

Write a migration that [CHANGE], following the migration conventions in this project. It must be safe to run against a live database with existing data. Include the rollback. Tell me explicitly whether it requires downtime and why.

107. Plan a zero-downtime schema change

Use when: You cannot take the system offline.

I need to [SCHEMA CHANGE] without downtime. Give me the expand-and-contract sequence: each deploy step, what code must be live before each migration, and the point of no return. Say what happens if we stop halfway.

108. Backfill existing data

Use when: New field, old rows.

Write a backfill for [DATA CHANGE]. It must be resumable, batched so it does not overwhelm the database, safe to run more than once, and observable so I can see progress. Include how to verify it completed correctly.

109. Port between languages or frameworks

Use when: Rewriting a component.

Port [FILE OR MODULE] from [SOURCE] to [TARGET]. Preserve the behaviour exactly, including edge cases, rather than writing what the code appears to intend. Where the target language makes a different approach clearly better, flag it instead of silently changing it.

110. Replace a dependency

Use when: *Swapping libraries.*

Replace [OLD LIBRARY] with [NEW LIBRARY] throughout this project. Map the API differences first and show me that mapping. Flag anywhere the new library has no equivalent for what we currently rely on.

111. Remove a deprecated feature

Use when: *Sunsetting something.*

We are removing [FEATURE]. Find everything connected to it: code, routes, database columns, config, feature flags, tests, documentation and scheduled jobs. Give me a removal plan ordered so nothing breaks mid-way, and note anything external that may still depend on it.

SECTION 13

DevOps, CI and Infrastructure

Pipelines, containers and config are code too, and they benefit from the same treatment.

112. Diagnose a CI failure

Use when: Red build, works locally.

The CI job [JOB NAME] is failing with [ERROR]. It passes locally. Compare the CI environment to local: versions, environment variables, available services, caching, file system assumptions and ordering. Find the difference that explains it.

113. Set up a CI pipeline

Use when: New project or missing automation.

Set up a CI pipeline for this project using [PLATFORM]. It should install dependencies with caching, lint, type-check, run tests, and build. Fail fast on the cheap checks. Explain each step and roughly how long it should take.

114. Speed up the build

Use when: CI is the bottleneck.

Our CI takes [DURATION]. Analyse the pipeline and find where the time goes. Look for missing caching, sequential steps that could run in parallel, redundant work, and oversized containers. Rank the fixes by time saved.

115. Review a Dockerfile

Use when: Container hygiene.

Review this Dockerfile for image size, build cache efficiency, security, and reproducibility. Look for: unpinned base images, secrets in layers, running as root, unnecessary build tools in the final image, and layer ordering that defeats caching.

116. Review infrastructure as code

Use when: Terraform, CloudFormation or similar.

Review the infrastructure code in [PATH]. Look for overly permissive access rules, resources without limits or budgets, missing encryption, no backup or retention policy, and anything that would be hard to recreate from scratch. Explain the risk of each finding.

117. Add health checks and monitoring

Use when: *Production readiness.*

Add health and readiness checks to [SERVICE] that reflect whether it can actually serve traffic, not just whether the process is alive. Then tell me the three metrics and two alerts I should have for this service and why those specifically.

118. Write a deployment checklist

Use when: *A risky release.*

Write a deployment checklist for [CHANGE]. Cover: pre-deploy verification, the deploy order if multiple components are involved, what to watch immediately after, the rollback procedure, and the specific signals that mean roll back now.

119. Audit environment configuration

Use when: *Config drift.*

List every environment variable and configuration value this project reads, where it is read, whether it has a default, and what happens if it is missing. Flag anything that fails silently or falls back to an unsafe default.

SECTION 14

Data, APIs and Integrations

Work at the edges of your system, where the assumptions are usually wrong.

120. Write a resilient API client

Use when: Calling an external service.

Write a client for [API]. Include timeouts on every request, retry with exponential backoff on the errors that are worth retrying and only those, explicit rate limit handling, typed responses, and clear errors when the API misbehaves. Make it easy to fake in tests.

121. Handle a flaky dependency

Use when: The third party is unreliable.

[SERVICE] is unreliable. Make our code degrade gracefully when it is slow or down: timeouts, circuit breaking, fallback behaviour, and a clear user-facing message. Tell me what functionality we can preserve without it and what we genuinely cannot.

122. Write a data transformation

Use when: Moving data between shapes.

Write a transformation from [SOURCE FORMAT] to [TARGET FORMAT]. Handle missing fields, type mismatches, and unexpected values explicitly rather than letting them through. Tell me what it does with a record it cannot transform.

123. Validate an incoming payload

Use when: Consuming external data.

Write validation for [PAYLOAD] entering our system from [SOURCE]. Validate structure, types, ranges and business rules. Reject with a specific message that says which field failed and why. Do not trust anything about the shape of the input.

124. Build a webhook receiver

Use when: Accepting callbacks.

Build a webhook endpoint for [SERVICE]. Verify the signature, respond quickly and process asynchronously, handle duplicate deliveries idempotently, and handle events arriving out of order. Log enough to reconcile against the sender's records.

125. Write a data import script

Use when: *One-off or recurring bulk load.*

Write a script to import [DATA] from [SOURCE]. It should validate before writing, process in batches, be resumable after a failure, report what it skipped and why, and support a dry-run mode. Assume the input data is messier than the sample.

126. Reconcile two data sources

Use when: *Two systems disagree.*

Write a reconciliation between [SOURCE A] and [SOURCE B]. Report records present in one and not the other, records that exist in both but differ, and the specific fields that differ. Do not change any data, just report.

127. Add pagination correctly

Use when: *Endpoints returning growing lists.*

Add pagination to [ENDPOINT]. Recommend offset or cursor based for this case and say why. Handle the edge cases: items added or deleted between pages, stable sort ordering, and the final page. Update the client code that consumes it.

SECTION 15

Agent Setup, Rules and Automation

Time spent configuring the agent pays back on every later prompt.

128. Generate the project rules file

Use when: First serious session in a repo.

Explore this codebase and write a rules file for coding agents. Include: what the project is, the stack, the directory conventions, the commands to build, test and lint, the code style rules that are not enforced by tooling, and the mistakes someone new to this codebase would make. Keep it factual and short enough to be read every session.

129. Improve an existing rules file

Use when: The agent keeps making the same mistakes.

Here is our current agent rules file: [PASTE OR PATH]. Based on the codebase, tell me what is wrong, out of date, or missing. Pay attention to conventions the code follows that the file does not mention.

130. Turn a repeated task into a command

Use when: You have typed the same prompt three times.

I keep asking for this: [TASK DESCRIPTION]. Turn it into a reusable custom command or skill for [AGENT], with the right parameters and a description that will make it trigger at the right moments. Show me the file and where it goes.

131. Write a self-checking prompt

Use when: You want the agent to verify its own work.

I want to ask an agent to [TASK]. Rewrite my request so it includes a verification step the agent can actually run to prove the work is correct, and so it stops and asks rather than guessing when something is ambiguous.

132. Automate a repetitive edit

Use when: Same change, many places, mechanically.

Write a script that performs [CHANGE] across this codebase. It should be idempotent, print what it changed, support a dry run, and refuse to run on a dirty working tree. Then run it in dry-run mode and show me the output.

133. Build a project scaffold generator

Use when: *You keep creating the same file structure.*

Every time we add a new [THING], we create the same set of files. Read two or three existing examples, work out the pattern, and write a generator that produces it correctly, including the tests and the registration steps people forget.

134. Set up a pre-commit gate

Use when: *Catching problems before they reach CI.*

Set up pre-commit checks for this project: formatting, linting, type checking, and secret detection. Keep the whole thing under a few seconds by only checking staged files. Tell me how to bypass it deliberately when needed.

135. Review your own agent workflow

Use when: *Periodic tune-up.*

Based on this session, tell me where my prompts were unclear, where you had to guess, and what context you needed that you did not have. Suggest specific changes to my rules file or my prompting that would have made this go better.

One last thing

The prompt is only half the work. The other half is the context your agent already has: a good rules file, a test command it can run, and a repository whose conventions are visible in the code. Fix those once and every prompt in this library gets better.